CP03 PROGRAMMING & SOFTWARE DEVELOPMENT

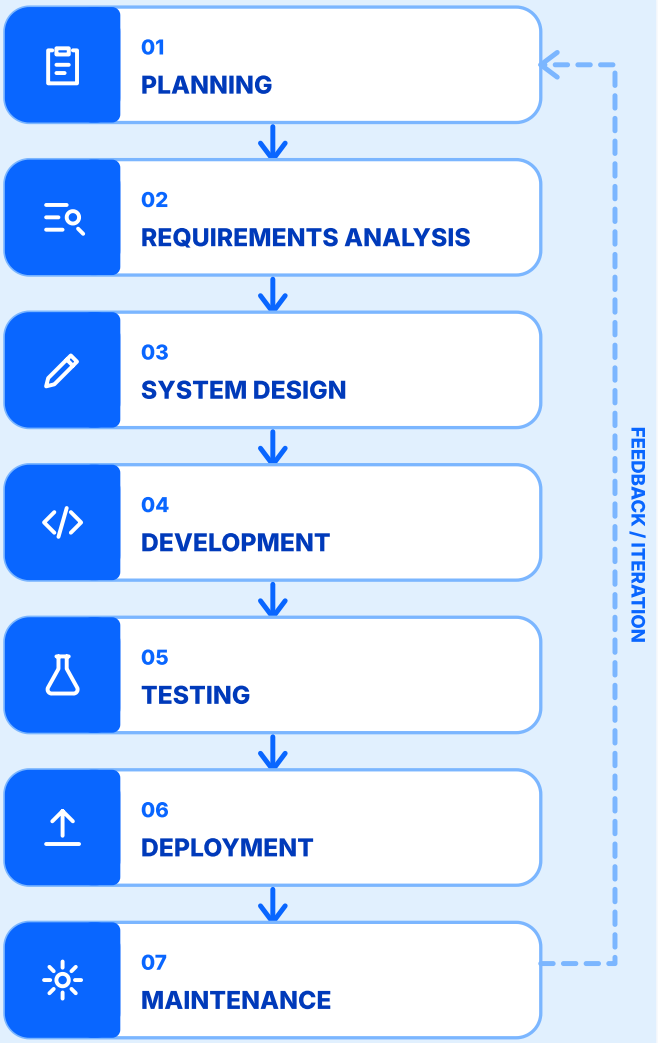
# Software Development Lifecycle

## SDLC Cheat Sheet: 7 Essential Phases

From planning an idea to maintaining a working software product.

## The 7 Phases At A Glance

Anak panah putus-putus: maklum balas dan pengulangan.




**APA ITU SDLC?**

**Software Development Life Cycle (SDLC)** ialah satu framework yang menerangkan aktiviti utama dalam proses merancang, membangunkan, menguji, melancarkan dan menyelenggara software.

## Why SDLC Matters

01 **Structured process**
Kerja software disusun mengikut aktiviti yang jelas.

02 **Understand requirements**
Keperluan pengguna difahami sebelum membina.

03 **Planning and resources**
Masa, bajet dan sumber dirancang lebih awal.

04 **Manage risks and quality**
Masalah dikesan dan dikawal lebih awal.

05 **Team coordination**
Stakeholders dan development team bekerja dengan rujukan sama.

06 **Basis for maintenance**
Dokumentasi dan rekod membantu kerja selepas release.

**SIAPA BIASANYA TERLIBAT?**

Users / clients · Project manager · Business analyst · UI/UX designer · Developers · Testers / QA · Operations / IT support

**NOTA**

Ini ialah satu model pengajaran yang biasa digunakan. Susunan dan pengulangan aktiviti boleh berbeza mengikut SDLC model dan projek.

## 01 PHASE 01 Planning

**Menentukan masalah yang ingin diselesaikan, skop projek, kebolehlaksanaan, bajet, jadual dan sumber.**

### MAIN OBJECTIVES
- Define project goals
- Set the project scope
- Check feasibility
- Estimate time, cost and resources

### COMMON ACTIVITIES
- Identify the problem
- Study stakeholders
- Estimate budget and timeline
- Assess risks
- Prepare project plan

### TYPICAL DELIVERABLES
- Project proposal
- Feasibility study
- Project plan

### PLANNING QUESTIONS
- **What problem are we solving?**
- **Who will use the system?**
- **What is the estimated budget?**
- **Can we deliver it with available resources?**

### EXAMPLE: ONLINE STUDENT ATTENDANCE SYSTEM
**Universiti mahu sistem untuk merekod kehadiran pelajar secara online supaya pensyarah tidak perlu guna kertas.**




**MEMORY TIP** Planning = Decide what to build and whether it is feasible.

## 02 PHASE 02 Requirements Analysis

**Mengumpul dan menganalisis apa yang pengguna dan sistem perlukan sebelum design bermula.**

### MAIN OBJECTIVES
- Gather user needs
- Separate functional and non-functional needs
- Remove unclear or conflicting requirements

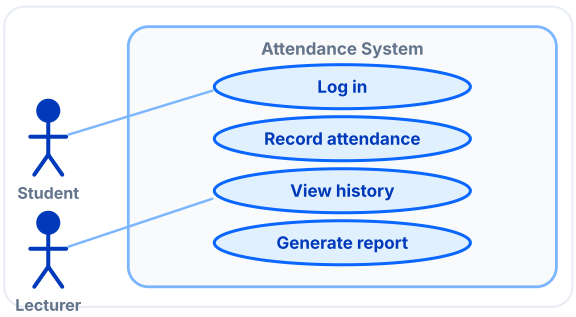
### COMMON ACTIVITIES
- Interviews and surveys
- Observe current process
- Write use cases and user stories
- Review with stakeholders

### TYPICAL DELIVERABLES
- Software Requirements Specification (SRS)
- Use cases
- User stories

### FUNCTIONAL (APA SISTEM BUAT)
- Student can log in
- Lecturer records attendance
- Student views attendance history
- Admin generates reports

### NON-FUNCTIONAL (SEJAUH MANA BAIK)
**Performance** **Security** **Reliability** **Usability** **Scalability** **Availability**

### EXAMPLE: ONLINE STUDENT ATTENDANCE SYSTEM
**Pelajar perlu log masuk dan melihat rekod kehadiran, pensyarah merekod kehadiran, dan admin menjana laporan.**




**MEMORY TIP** Requirements = Understand what users and the system need.

# 03 PHASE 03 System Design

**Menentukan bagaimana software akan dibina berdasarkan requirements yang telah dipersetujui.**

### MAIN OBJECTIVES
- Translate requirements into a technical plan
- Choose architecture and structure
- Plan data and interface

### COMMON ACTIVITIES
- System architecture
- Database design
- User interface design
- Component design
- Security design

### TYPICAL DELIVERABLES
- Architecture diagram
- ERD
- Database schema
- Wireframes / mockups
- Technical design specifications

### 5 DESIGN ACTIVITIES

**System architecture**
Struktur utama dan hubungan antara bahagian sistem.

**Database design**
Entiti, hubungan dan struktur data.

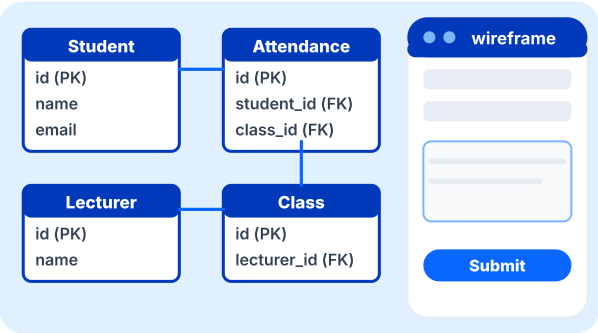
**UI design**
Skrin, layout dan aliran pengguna.

**Component design**
Tugas dan interface setiap modul.

**Security design**
Kawalan akses dan perlindungan data.

### EXAMPLE: ONLINE STUDENT ATTENDANCE SYSTEM

**Modul dirancang: Student, Lecturer, Class, Attendance dan Report. Jadual database dan skrin log masuk dilakar dahulu.**




**MEMORY TIP** Design = Decide how the software will be built.

# 04 PHASE 04 Development / Implementation

**Menukar design kepada code dan modul yang berfungsi.**

### MAIN OBJECTIVES
- Write working code
- Build modules
- Integrate components

### COMMON ACTIVITIES
- Write and review code
- Use version control
- Build database and APIs
- Unit-test own code
- Integrate modules

### TYPICAL DELIVERABLES
- Source code
- Software modules
- Database implementation

### TYPICAL DEVELOPMENT FLOW

Write code → Commit to Git → Code review → Build → Integrate

### EXAMPLE: ONLINE STUDENT ATTENDANCE SYSTEM

**Pasukan membina modul log masuk, fungsi rekod kehadiran dan paparan sejarah kehadiran.**

### USEFUL TOOLS

VS Code | Git | GitHub | Languages / frameworks | DBMS

attendance.js

```
async function recordAttendance(studentId, classId) {
const record = {
studentId, classId, status: "present"
};
return db.attendance.insert(record);
}
```

Git: main | 0 errors

**MEMORY TIP** Development = Turn the design into working code.

## 05 PHASE 05 Testing

**Menyemak sama ada software berfungsi seperti dijangka dan mencari defect sebelum release.**

### MAIN OBJECTIVES
- Find defects
- Verify expected behaviour
- Gather quality evidence

### COMMON ACTIVITIES
- Write test plan and test cases
- Run tests
- Report defects
- Retest after fixes

### TYPICAL DELIVERABLES
- Test plan
- Test cases
- Test reports
- Defect reports

**1 Unit testing**
Menguji unit kecil seperti satu function.

**2 Integration testing**
Menguji interaksi antara modul yang digabungkan.

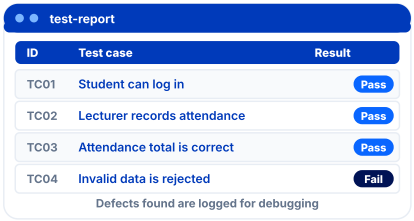
**3 System testing**
Menguji sistem lengkap sebagai satu keseluruhan.

**4 Acceptance testing**
Pengguna atau client mengesahkan sistem memenuhi keperluan.

### OTHER TESTING CATEGORIES
Performance | Security | Usability | Regression

**IMPORTANT**
Testing does not guarantee software is completely defect-free. It provides evidence about quality and helps identify problems.

### EXAMPLE: ONLINE STUDENT ATTENDANCE SYSTEM
**Adakah pelajar boleh log masuk? Adakah jumlah kehadiran dikira dengan betul? Adakah data tidak sah ditolak?**




**MEMORY TIP** Testing = Check whether the software works as expected.

## 06 PHASE 06 Deployment

**Menjadikan software tersedia kepada pengguna dalam production environment.**

### MAIN OBJECTIVES
- Release to users safely
- Keep downtime low
- Confirm the release works

### COMMON ACTIVITIES
- Prepare production environment
- Migrate data
- Configure servers
- Release the build
- Monitor after release
- Prepare rollback plan

### TYPICAL DELIVERABLES
- Released software
- Production environment
- Release notes

### DEPLOYMENT APPROACHES
**Manual**
Dibuat langkah demi langkah oleh orang.

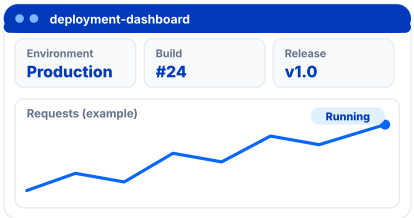
**Automated**
Guna script atau tool.

**CI/CD**
Build, test dan release melalui pipeline.

**Phased / staged rollout**
Release kepada kumpulan pengguna secara berperingkat.

**Blue-green**
Dua persekitaran; trafik ditukar apabila versi baharu sedia.

### EXAMPLE: ONLINE STUDENT ATTENDANCE SYSTEM
**Sistem kehadiran dilancarkan dahulu kepada satu fakulti sebelum semua pelajar menggunakannya.**




**MEMORY TIP** Deployment = Make the software available to users.

# 07 PHASE 07 Maintenance

**Memastikan software terus berguna, selamat dan boleh dipercayai selepas dilancarkan.**

### MAIN OBJECTIVES
- Fix problems found after release
- Keep software compatible
- Improve and reduce future risk

### COMMON ACTIVITIES
- Monitor and collect feedback
- Fix bugs
- Apply patches and updates
- Update documentation

### TYPICAL DELIVERABLES
- Patches
- Updates
- Bug fixes
- Maintenance documentation

### 4 CATEGORIES OF MAINTENANCE

**CORRECTIVE**
**Betulkan bug atau error.**
Fix incorrect attendance report totals.

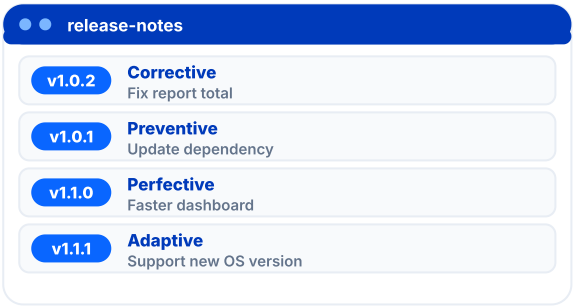
**ADAPTIVE**
**Sesuaikan dengan perubahan persekitaran.**
Update for a new operating system version or external API.

**PERFECTIVE**
**Tambah baik prestasi atau ciri.**
Reduce dashboard load time or add a reporting feature.

**PREVENTIVE**
**Kurangkan risiko masalah akan datang.**
Refactor code or update vulnerable dependencies.

### EXAMPLE: ONLINE STUDENT ATTENDANCE SYSTEM

**Selepas semester bermula, pelajar melaporkan jumlah kehadiran salah. Pasukan membetulkan dan mengeluarkan patch.**




**MEMORY TIP** **Maintenance = Keep software useful and reliable after release.**

## Deliverable Quick Reference | Artefak biasa bagi setiap phase.

| PHASE | COMMON DELIVERABLES |
|---|---|
| **01** Planning | Project proposal, feasibility study, project plan |
| **02** Requirements | SRS, use cases, user stories |
| **03** Design | Architecture diagram, ERD, wireframes, technical specifications |
| **04** Development | Source code, software modules, database implementation |
| **05** Testing | Test plan, test cases, test reports, defect reports |
| **06** Deployment | Released software, production environment, release notes |
| **07** Maintenance | Patches, updates, bug fixes, maintenance documentation |

**NOTA**
Deliverables vary depending on project size, organisation, development approach and documentation requirements.

# Waterfall vs Agile | Dua cara menyusun aktiviti SDLC.

**SDLC menerangkan aktiviti pembangunan. Waterfall dan Agile ialah pendekatan yang menyusun dan mengurus aktiviti itu dengan cara berbeza. Kedua-duanya boleh melibatkan aktiviti SDLC yang sama.**

## Waterfall

Sequential, phase-oriented




Pendekatan yang menyusun kerja secara lebih berturutan, dengan setiap phase mempunyai output yang dirujuk oleh phase seterusnya.

### CHARACTERISTICS

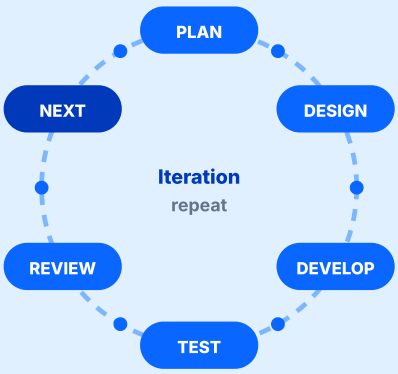
- Phases follow a planned sequence
- Documentation is usually emphasised
- Requirements are defined early
- Review points between phases

### SUITABLE CONTEXT

Requirements stabil dan jelas, projek dengan keperluan dokumentasi atau compliance yang ketat.

## Agile

Iterative, incremental




Pendekatan yang membahagikan kerja kepada kitaran pendek. Setiap iteration menghasilkan increment yang boleh dikaji dan diberi maklum balas.

### CHARACTERISTICS

- Work is delivered in short cycles
- Feedback is collected frequently
- Priorities can be adjusted between iterations
- Close collaboration with stakeholders

### SUITABLE CONTEXT

Requirements berkemungkinan berubah, pengguna boleh memberi feedback awal dan berkala.

## Comparison

| | WATERFALL | AGILE |
|---|---|---|
| **Workflow** | More sequential and phase-oriented | Iterative and incremental cycles |
| **Planning** | Detailed plan usually prepared early | Planned continuously, in short cycles |
| **Requirements** | Usually defined early | Can be refined between iterations |
| **Feedback** | Often at review points and later stages | Frequent, throughout the project |
| **Delivery** | Often delivered at the end or in major releases | Delivered in increments |
| **Change** | Changes handled through a more formal process | Changes are expected and managed each iteration |

**IMPORTANT CLARIFICATION**

Agile bukan satu methodology tunggal. Scrum dan Kanban ialah contoh framework atau pendekatan yang dikaitkan dengan cara kerja Agile. Scrum tidak sama dengan Agile. Tiada satu pendekatan yang sesuai untuk semua projek.

## SDLC Quick Revision | 7 phases dalam satu baris.

| | | | |
|---|---|---|---|
| **Planning** | What should we build? | **Requirements** | What do users need? |
| **Design** | How will we build it? | **Development** | Build the software. |
| **Testing** | Does it work correctly? | **Deployment** | Release it to users. |
| **Maintenance** | Keep it working and improve it. | | |

## Commonly Confused Pairs

**FUNCTIONAL VS NON-FUNCTIONAL**
Functional: apa yang sistem buat. Non-functional: sejauh mana baik sistem itu berfungsi, seperti security dan performance.

**VERIFICATION VS VALIDATION**
Verification: adakah kita bina produk dengan betul mengikut spesifikasi. Validation: adakah kita bina produk yang betul untuk keperluan pengguna.

**TESTING VS DEBUGGING**
Testing: mengesan defect. Debugging: mencari punca dan membetulkan defect itu.

**DEVELOPMENT VS DEPLOYMENT**
Development: membina software. Deployment: melancarkan software kepada pengguna.

## Memory Technique

**PLAN → REQUIREMENTS → DESIGN → DEVELOP → TEST → DEPLOY → MAINTAIN**
Plan it, understand it, design it, build it, test it, release it and maintain it.

## Self-Test

**Q1** **Which phase focuses on understanding what users and the system need?**
**Jawapan:** Requirements Analysis

**Q2** **Which phase decides how the software will be built?**
**Jawapan:** System Design

**Q3** **What is integration testing used for?**
**Jawapan:** To evaluate interactions between integrated components or modules

**Q4** **A bug in report totals is fixed after release. Which maintenance type is this?**
**Jawapan:** Corrective Maintenance

**Q5** **How do Waterfall and Agile generally differ?**
**Jawapan:** Waterfall generally follows a more sequential phase-oriented approach while Agile organises work through iterative and incremental cycles.

**SDLC helps teams build software systematically, from the first idea to long-term maintenance.**
FINAL TAKEAWAY

**Download untuk rujukan assignment atau exam.**
SDLC ialah model pengajaran; amalan sebenar berbeza mengikut projek.

**yusufaimxn**
Technology, Made Understandable.